

MPC Controller: End-to-End Workflow

From MATLAB algorithm to FPGA validation, open loop and closed loop

Target: Cora Z7-07S (Vivado / Vitis 2022.1)

Verified, open-loop replay: 801 / 801 samples match MATLAB bit-exactly (100.00%), 739.8 ns average FPGA solve time per step.

Verified, closed loop on hardware: with the plant on the PS and the MPC IP in the loop over AXI, all 801 command pairs and quantized states from the Cora match the MATLAB simulation exactly, and the two trajectories agree to the UART print resolution (position difference under $1e-6$ m at every step). Both sides count 1.80 laps.

Overview

This document is the complete reproduction procedure for the MPC controller pipeline: the MATLAB algorithm and its testbench, HDL integration into a custom Vivado IP block design (Zynq processing system, AXI interconnect, MPC IP), Vitis bare-metal application development, FPGA execution, and validation against the MATLAB simulation. Validation now runs in two forms. The open-loop replay streams recorded trajectory states through the FPGA and checks every returned command against the MATLAB-expected value. The closed-loop harness goes further: it integrates the vehicle plant itself, feeds the live state to the controller each step, and applies whatever the controller returns, so any disagreement compounds instead of resetting at the next sample.

A single MATLAB testbench run generates every data artifact (trajectory states, expected commands, the C header, the VHDL simulation package, the closed-loop track header). This single source of truth removes drift between MATLAB simulation, VHDL behavioral simulation, and FPGA execution.

Pipeline at a glance: `fcs_mpc_v2.m` -> `fcs_mpc_v2_fixpt.vhd` -> custom IP -> bitstream -> Cora FPGA -> `fpga_log.csv` / `c_closed_loop.csv` -> MATLAB comparison scripts

Final result summary

Open-loop replay, FPGA vs MATLAB:

```
matches      : 801 / 801 (100.00%)
accel diffs  : 0    max|err|=0
steer diffs  : 0    max|err|=0
PL-only latency (ns): min=723   avg=739.8   max=806
End-to-end latency (ns): min=3720 avg=3752.7 max=4387
```

Closed loop, Cora FPGA backend vs MATLAB (801 rows each):

```
accel diffs : 0    steer diffs : 0
quantized state and reference columns identical at every step
max position difference: 6.9e-7 m (the UART 6-digit print resolution)
laps: 1.80 both sides    err-column peak: 1.008 m both sides
```

PL-only latency is the time the IP spends solving the MPC problem. End-to-end latency adds the AXI transfer overhead: six input writes, one kick, status polling, two output reads.

Required files

File	Role
<code>fcs_mpc_v2.m</code>	Controller algorithm (MATLAB source of truth)
<code>fcs_mpc_v2_tb3.m</code>	Closed-loop MATLAB testbench. Generates all data artifacts.
<code>fcs_mpc_v2_fixpt.vhd</code>	Hand-written VHDL controller, bit-exact to the .m
<code>MPC_controller_AXI_v1_0.vhd</code>	AXI4-Lite IP top wrapper (boilerplate)
<code>MPC_controller_AXI_v1_0_S00_AXI.vhd</code>	AXI slave with register map, FSM, and DUT instance
<code>trajectory_data_pkg.vhd</code>	VHDL package with trajectory data (simulation only)
<code>fcs_mpc_v2_fixpt_tb.vhd</code>	Self-checking VHDL testbench (simulation only)
<code>mpc_smoketest.c</code>	Bare-metal open-loop replay app, streams CSV to UART
<code>trajectory_data.h</code>	C header with the recorded trajectory state arrays
<code>mpc_closed_loop.c</code>	Closed-loop validation harness (host SW backend or Cora FPGA backend)
<code>reference_track.h</code>	Auto-generated track points and simulation parameters for the closed loop
<code>compare_fpga_matlab.m</code>	MATLAB open-loop validation script (figures and stats)

Stage 1: MATLAB algorithm and testbench

1.1 The controller (fcs_mpc_v2.m)

The controller is a kinematic-model MPC. It enumerates 17 steering candidates by 8 acceleration candidates (136 candidates per sample), evaluates a position-tracking plus steering-effort plus speed-reward cost for each, and outputs the pair with minimum cost.

Parameter	Value
Position scale (SF_POS)	64 units per meter
Heading scale (SF_PSI)	41 units per radian (about 256 / 2pi)
Velocity scale (SF_V)	64 units per m/s
Time step (Ts)	0.1 s
STEER_OPTS (17 values)	{0, +/-2, +/-6, +/-10, +/-14, +/-18, +/-22, +/-24, +/-26}
ACCEL_OPTS (8 values)	{0, 10, 5, 1, -5, -1, -10, -20}
W_ERR / W_STEER / W_SPEED	15 / 2 / 2
CLAMP_VAL (axis-error clamp)	1000 (about 15 m)
V_MAX	320 (5 m/s)

Function signature: `function [accel_cmd, steer_cmd] = fcs_mpc_v2(x, y, psi, v, ref_x, ref_y)`. Inputs are int16 fixed point. Outputs are int16 sfix6 (accel) and sfix6 (steer). `accel_cmd` was widened from sfix5 to sfix6 to fit the -20 member of `ACCEL_OPTS`, which does not fit 5-bit two's complement (-16..15). This widening is the single most disruptive change to apply consistently across the .m, the VHDL DUT, and the AXI wrapper. Details in Appendix A.2.

1.2 Cost function structure

The controller is a finite-control-set (FCS) MPC: it does not solve a QP. It enumerates the 136 candidate (steer, accel) pairs and picks the one that minimizes a single-step cost with three additive terms:

```
cost      = dist_cost + steer_cost - speed_rwd
dist_cost = (|x_next - ref_x|_clamped + |y_next - ref_y|_clamped) * W_ERR
steer_cost = |delta| * W_STEER
speed_rwd  = v_next * W_SPEED
```

dist_cost penalizes the predicted distance from the lookahead reference one step ahead. The L1 (Manhattan) norm is used instead of L2 because it avoids a multiplication and saves cycles. Each axis is clamped to `CLAMP_VAL=1000` before weighting. **steer_cost** penalizes large steering angles; the small `W_STEER` against `W_ERR` means the controller accepts large steering when tracking demands it but breaks ties toward straight. **speed_rwd** rewards higher speed and is subtracted from the cost. It is the only mechanism pushing the controller forward: without it, the optimal action at `v=0` with the reference straight ahead would be to do nothing. The negative sign makes the controller accelerate.

1.2.1 Why these weights, and why the clamp

The 15:2:2 ratio biases the cost toward tracking accuracy first. At the per-axis clamp limit (1000 units, about 15 m off track), `dist_cost` alone is $(1000+1000) * 15 = 30000$, about 90% of the int16 ceiling (32767). `steer_cost` peaks at $26 * 2 = 52$ and `speed_rwd` at $320 * 2 = 640$, so tracking error is weighted roughly 60 times more heavily than effort or speed. That is why the lap converges to within 1 m even at saturated velocity. Without the clamp, `dist_cost` could exceed 32767 and wrap negative, making a far-off-track candidate look optimal. `CLAMP_VAL=1000` keeps 30000 under the ceiling with headroom, and it also flattens the gradient far from the reference, keeping the controller responsive to local features instead of always flying straight at a distant point.

1.2.2 Tie breaking

Multiple candidates can produce identical cost, especially early in the run when `v=0` makes the velocity update trivial. The min-cost selection uses strict less-than (`cost < min_cost`), so the first minimum encountered wins. With the loop order (steer outer, accel inner) and both option arrays starting at 0, ties break toward `delta=0`, `accel=0`. This deterministic tie break is essential for MATLAB to VHDL bit-exactness; see Stage 3.

1.3 Quantization scheme

Every signal uses a fixed-point representation with a small, hand-tuned bit width. There is no floating point. The scale factors are `SF_POS = 64` (1 m = 64 units, Q10.6: 10 integer bits for a +/-512 m range, 6 fractional bits for about 1.5 cm resolution), `SF_PSI = 41` (2pi maps to about 256, so one LUT bin is 0.0245 rad, about 1.4 degrees), and `SF_V = 64` (same Q10.6 resolution as position; `V_MAX = 320 = 5 m/s`).

Signal	Width	Range (units)	Range (physical)
x	sfix14	+/-8192	+/-128 m

Signal	Width	Range (units)	Range (physical)
y	sfix13	+/-4096	+/-64 m
psi	sfix9	+/-256	+/-2pi rad (full circle)
v	ufix9	0..511	0..8 m/s (capped at 5 m/s)
ref_x	sfix14	+/-8192	+/-128 m
ref_y	ufix12	0..4095	0..64 m
accel_cmd	sfix6	+/-32	carries ACCEL_OPTS {-20..10}
steer_cmd	sfix6	+/-32	carries STEER_OPTS {-26..26}
cost (internal)	sfix16	+/-32767	weighted dist + steer - speed

The widths come from the worst-case excursion of each signal under the closed-loop dynamics, then the smallest width that still fits with margin. Smaller widths mean fewer LUT and DSP resources and shorter combinational paths. The widths are visible in the AXI register slicing and in the entity ports of fcs_mpc_v2_fixpt.vhd.

1.3.1 Trigonometry: SIN_LUT and COS_LUT

Real sin/cos would need a CORDIC pipeline or a polynomial with multipliers; both are large. The algorithm instead uses two 256-entry, 8-bit signed amplitude LUTs. SIN_LUT is built as [Q1, Q2, Q3, Q4] from a 64-entry first quadrant; COS_LUT is the same table shifted by 64 entries (pi/2). Each entry is the value scaled by 128 and rounded. The LUT index is the low 8 bits of psi_next, which exploits the natural modulo-256 wrap to handle full rotations. Sixty-four entries per quadrant fit cleanly in 8 bits with the folding scheme, and 1.4 degrees of angular resolution is enough: the smallest non-zero steering step is 2 angular units (2.86 degrees), so a finer LUT would not change the optimal selection.

1.3.2 Arithmetic-shift-right scalings

Operation	Why this shift count
dpsi = (v * delta) >> 8	v is Q10.6, delta is the raw steer option. Dividing by 256 converts to PSI scale and folds in the implicit Ts (v*delta*Ts/L is about v*delta/256 for L=2.5 m, Ts=0.1 s at these scales). One shift bundles both.
v_red = v >> 4	Pre-scales velocity by 1/16 before multiplying with the 8-bit LUT amplitude, keeping the product in int16 range while preserving precision.
dx = (v_red*c) >> 5, dy = (v_red*s) >> 5	v_red has scale 4, the LUT amplitude has scale 1/128, so the product has scale 1/32. The final shift by 5 brings the delta to position scale (1 m = 64).

The shift counts are pre-multiplied so the kinematic update produces position deltas already in Q10.6. Changing any of them silently is the easiest way to break the controller: the search still finds a best candidate, but best in the wrong units.

1.4 The MATLAB testbench (fcs_mpc_v2_tb3.m)

The testbench drives a bicycle-model plant, calls fcs_mpc_v2 each step, integrates one step of plant dynamics, and accumulates the trajectory. It is also the source-of-truth generator for all downstream data artifacts. (It still prints 'Auto-generated by fcs_mpc_v2_tb2.m' in the trajectory_data.h comment; the name is left over from an earlier iteration and is cosmetic only.) The per-step loop, simplified:

```

for k = 1:length(time)
    % Nearest point on reference track
    [~, min_idx] = min((ref_path_x - x_phys).^2 + (ref_path_y - y_phys).^2);
    % Dynamic lookahead (0.8 m at low speed, more at high speed)
    lookahead_dist = max(0.8, 0.25 * v_phys);
    [r_x, r_y] = walk_ahead(ref_path, min_idx, lookahead_dist);
    % Pack state to int16 with scale factors
    x_in = int16(x_phys * SF_POS);
    y_in = int16(y_phys * SF_POS);
    psi_in = int16((mod(psi_phys + pi, 2*pi) - pi) * SF_PSI);
    v_in = int16(v_phys * SF_V);
    rx_in = int16(r_x * SF_POS);
    ry_in = int16(r_y * SF_POS);
    % Run controller
    [acc_int, str_int] = fcs_mpc_v2(x_in, y_in, psi_in, v_in, rx_in, ry_in);
    % Plant integration (bicycle model)
    x_phys = x_phys + v_phys * cos(psi_phys) * Ts;
    y_phys = y_phys + v_phys * sin(psi_phys) * Ts;
    psi_phys = psi_phys + (v_phys / L_phys) * tan(str_cmd) * Ts;
    v_phys = (v_phys + acc_cmd) * 0.99;
end

```

One run writes eleven files: eight .dat files (x, y, psi, v, ref_x, ref_y, accel_cmd_expected, steer_cmd_expected; 4-digit lowercase hex of each int16 bit pattern, one sample per line), trajectory_data.h for the replay app, trajectory_data_pkg.vhd for the VHDL testbench, and reference_track.h for the closed-loop harness. The .dat values use truncated widths inside the 4 hex digits (x stores 14 bits, psi stores 9, and so on), which matters for the comparison scripts; see Appendix A.3. Console output after a run:

```
>> fcs_mpc_v2_tb3
Starting Simulation (Constraint Mode)...
Total Laps: 1.80 | Max Error: 0.987 m
Wrote 8 .dat files (801 samples each)
Wrote trajectory_data.h (801 samples)
Wrote trajectory_data_pkg.vhd (801 samples)
```

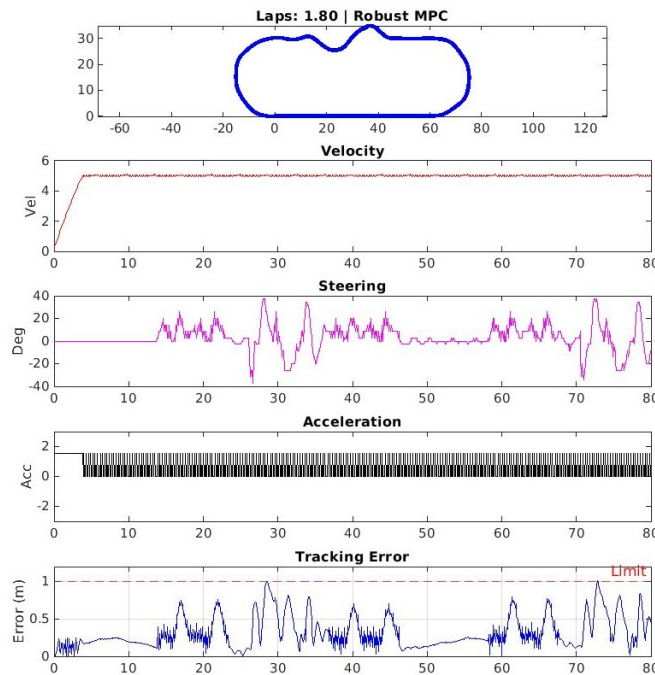


Figure 1.1: MATLAB closed-loop simulation. 1.80 laps completed, peak tracking error 0.987 m, under the 1 m limit.

1.5 Acceptance criteria for this stage

Before proceeding to Vivado, verify in MATLAB that: (1) Total Laps is at least 1.5, so the controller is actually tracking; (2) Max Error stays below 1.0 m; (3) velocity saturates at 5 m/s after the initial transient; (4) steering shows back-and-forth corrections at corners; (5) acceleration settles to small values once velocity saturates. If any criterion fails, fix the cost weights, option sets, or initial conditions in fcs_mpc_v2.m before continuing. The FPGA can only reproduce what MATLAB produces, and debugging in hardware costs an order of magnitude more time than debugging in MATLAB.

Stage 2: Vivado, custom IP creation and bitstream

This stage encapsulates the controller as an AXI4-Lite peripheral IP, instantiates it inside a Zynq block design, and generates a bitstream.

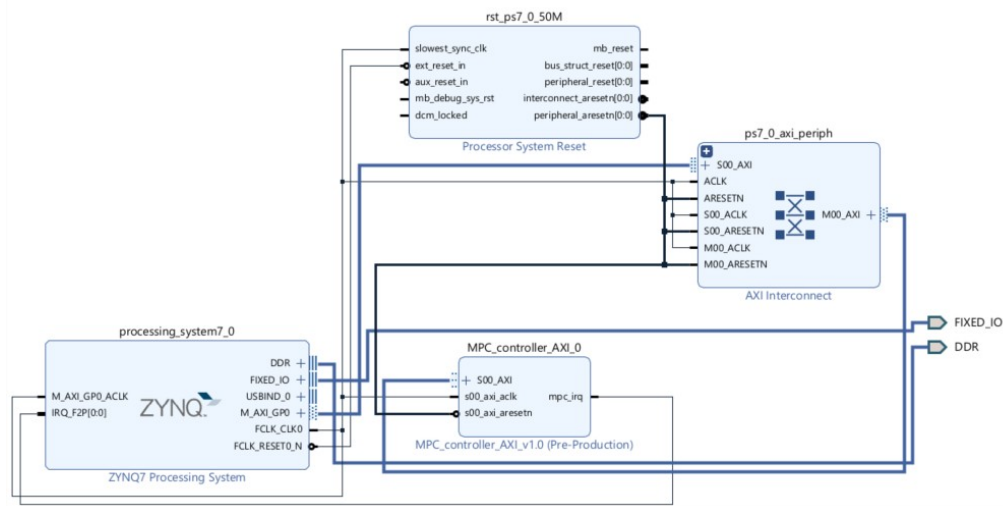


Figure 2.1: Target block design topology. ZYNQ7 PS to AXI Interconnect to MPC IP, with a shared 50 MHz clock and synchronous reset.

2.1 Create the Vivado project

1. Launch Vivado 2022.1 -> Create Project. Name: mpc_vhdl. Location: any folder outside OneDrive/Dropbox sync (file locks during synthesis are a known issue, Appendix A.7).
2. Project Type: RTL Project, check 'Do not specify sources at this time'. Default Part: xc7z007s-clg400-1, the Cora Z7-07S silicon. **Do not pick xc7z010**: a bitstream for the wrong part either fails to fit or programs but misbehaves.

2.2 Create the custom MPC IP

1. Tools -> Create and Package New IP -> Create a new AXI4 peripheral. Name: MPC_controller_AXI, version 1.0. IP location outside the Vivado project, e.g. C:\Users\manos\Downloads\result\ip_repo.
2. AXI Lite slave: name S00_AXI, mode Slave, data width 32, 20 registers (offsets 0x00-0x4C; the design uses through 0x40). Then 'Edit IP' -> Finish; a second Vivado instance opens.
3. Replace the wizard skeleton: in Package IP -> File Groups, remove the wizard's MPC_controller_AXI_v1_0_S00_AXI.vhd and add the project's three sources to the Synthesis and Simulation groups: fcs_mpc_v2_fixpt.vhd, MPC_controller_AXI_v1_0.vhd, MPC_controller_AXI_v1_0_S00_AXI.vhd.
4. Remove the 'Software Driver' file group (right-click -> Remove File Group). The wizard's empty C driver files break the Vitis BSP build with 'cc1.exe: fatal error: *.c: Invalid argument'. The application talks to the IP directly through Xil_In32 / Xil_Out32, so no driver layer is needed (Appendix A.10).
5. Review and Package, optionally bump the version so Vivado flags it as updated, then Re-Package IP and close the editor.

After packaging, the IP hierarchy must be exactly MPC_controller_AXI_v1_0 -> MPC_S00_AXI_inst -> u_mpc : fcs_mpc_v2_fixpt. In particular, mpc_axi_wrapper.vhd from earlier iterations must NOT be present; its slicing now lives inside the S00_AXI file. If it appears in File Groups, remove it (Appendix A.4).

2.3 Block design integration

1. Create Block Design 'design_1'. Add the ZYNQ7 Processing System, apply the board preset if available, otherwise set: M_AXI_GP0 enabled, FCLK_CLK0 = 50 MHz, IRQ_F2P[0:0] checked, the Cora DDR3 part confirmed, UART1 on MIO 48..49.
2. Register the IP repo (Settings -> IP -> Repository -> add ip_repo), add MPC_controller_AXI to the canvas, and Run Connection Automation. Vivado adds ps7_0_axi_periph and rst_ps7_0_50M and wires S00_AXI to M_AXI_GP0, clocks, resets, and mpc_irq to IRQ_F2P. The topology matches Figure 2.1.
3. Address Editor: assign MPC_controller_AXI_0 to 0x43C0_0000, size 64K. xparameters.h will then define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR as 0x43C00000. If the C app prints a different base address at boot, the address map is wrong.
4. Validate Design, then Create HDL Wrapper ('Let Vivado manage wrapper'). design_1_wrapper.vhd becomes the top synthesis entity.

2.4 Bitstream and hardware export

1. Generate Bitstream (5-10 minutes typical). Then sanity-check the timestamp: dir /T:W <project>\...\impl_1\ design_1_wrapper.bit must show the last few minutes. If older, Vivado reused a cached bitstream; Reset Runs on synth_1 and rebuild.
2. File -> Export -> Export Hardware, and **check 'Include bitstream'**. Without it the XSA has no .bit and Vitis silently keeps running the previous bitstream. Verify with unzip -l design_1_wrapper.xsa | grep .bit.

Stage 3: VHDL behavioral testbench

Before synthesizing an algorithm change, validate VHDL to MATLAB bit-equivalence in behavioral simulation. The VHDL testbench runs in seconds against the roughly ten minutes of a bitstream rebuild.

1. Add trajectory_data_pkg.vhd and fcs_mpc_v2_fixpt_tb.vhd under Simulation Sources -> sim_1, marked 'Simulation Only', not 'Design+Simulation'.
2. The TB drives 801 samples at one per 10 ns clock plus reset, about 8 us of simulated time; Vivado defaults to 1 us. Set xsim.simulate.runtime to 15us in Project Settings -> Simulation.
3. Run Behavioral Simulation and wait for the console summary.

```
[TB] Samples checked : 801
[TB] accel matches   : 801 / 801
[TB] steer matches   : 801 / 801
[TB] PASS -- all 801 samples match MATLAB bit-exactly.
```

Do not generate a bitstream until this passes. The FPGA only reproduces what the VHDL specifies; if VHDL and MATLAB disagree here, hardware will disagree too.

3.1 What makes bit-exactness possible

fcs_mpc_v2_fixpt.vhd is a hand-written translation of fcs_mpc_v2.m that preserves every numerically observable property, so both produce identical 16-bit outputs for any state vector. The non-obvious divergence points between MATLAB and VHDL semantics, and how each is handled:

- **Loop order and tie break.** The .m iterates steer outer, accel inner; the VHDL replicates this with the same FOR order, and both use strict cost < min_cost. Ties therefore break to the same first-encountered minimum. At low velocity many candidates tie, so a <= comparison or a different order would silently diverge on exactly those samples.
- **Arithmetic right shift.** MATLAB's bitsra on int16 and VHDL's shift_right on signed both preserve the sign bit and round toward minus infinity. A Python prototype verified equivalence on 5000 random state vectors plus the 801 trajectory states before any bitstream was built.
- **Multiplication width.** MATLAB int16 * int16 saturates to int16 with no widening; VHDL signed * signed produces a 32-bit result that is resized after the shift. The two agree as long as no intermediate exceeds int16. The largest product here is $v * \delta = 320 * 26 = 8320$, well inside range. If a weight or option change ever pushes an intermediate past 32767, the VHDL TB fails before the bug reaches hardware.
- **LUT indexing.** The .m uses bitand(psi_next, 255) + 1 with 1-based indexing; the VHDL uses to_integer(unsigned(psi_next(7 downto 0))) with 0-based indexing. For negative psi_next both produce the same low-8-bits-as-unsigned value (-30 is 0xFFE2; AND 0xFF gives 226 on both sides).
- **Unsigned promotion at the AXI boundary.** v and ref_y are ufix9 and ufix12. The DUT zero-extends them to signed(15:0), which is safe because both are non-negative by construction. MATLAB never faced this because it stored them as int16 from the start; the AXI slicing introduces the unsigned interpretation to save bus bits.
- **Registered I/O.** The .m is a single function call; the VHDL has an input register stage, a combinational body, and an output register stage, two cycles total. The behavioral TB accounts for this with a 2-sample shift in the checker.

Deliberately not identical, and fine: MATLAB iterates in a function while the VHDL evaluates all 136 candidates in parallel combinational logic; MATLAB uses double precision for the plant integration while the VHDL never sees the plant; MATLAB returns variables while the VHDL updates two registered ports each clock.

Stage 4: Vitis, bare-metal applications

4.1 Platform project

1. Vitis 2022.1 -> New Platform Project 'coraz7_07s_plat' from design_1_wrapper.xsa. OS standalone, processor ps7_cortexa9_0, 32-bit. Build it.
2. Verify the BSP picked up the IP: findstr /R "MPC" ... \bsp\... \include\parameters.h must show the base address macros. If no hits, the IP is missing from the BD address editor; return to Stage 2.3.

```
#define XPAR_MPC_CONTROLLER_AXI_0_DEVICE_ID 0
#define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_BASEADDR 0x43C00000
#define XPAR_MPC_CONTROLLER_AXI_0_S00_AXI_HIGHADDR 0x43C0FFFF
```

4.2 The two applications

Two application projects build on the same platform, both created from the 'Empty Application(C)' template. **mpc_smoketest** is the open-loop replay app: copy mpc_smoketest.c and trajectory_data.h into src/. **mpc_closed_loop** is the closed-loop harness: copy mpc_closed_loop.c and reference_track.h into src/. The closed-loop harness also compiles unchanged on a PC, where it selects the software controller port; Stage 7 covers it in detail.

The replay app supports three output modes via `-DOUTPUT_MODE` in C/C++ Build -> Settings -> Symbols: `MODE_SUMMARY` (0, final stats and edge samples), `MODE_VERBOSE` (1, per-sample table), `MODE_CSV` (2, pure CSV for `compare_fpga_matlab.m`). Use `OUTPUT_MODE=2` for end-to-end validation. Clean and build; expect `mpc_smoketest.elf` in Debug/.

Stage 5: Run on FPGA

1. Connect the Cora over USB; it enumerates as JTAG plus USB-UART. Find the COM port in Device Manager.
2. **Open the serial terminal before programming.** UART output starts the moment the FSBL hands off; a late terminal loses the banner and the first rows (Appendix A.6). Settings: 115200 8N1, no flow control. Enable session logging to a file.
3. Vitis: right-click the app -> Run As -> Launch on Hardware (System Debugger). Vitis programs the bitstream from the XSA, runs the FSBL, and launches the app; 10-15 seconds total.

Expected UART output for the replay app in `MODE_CSV`: a header line, 801 data rows, then footer comments.

```
idx,gx,gy,gpsi,gv,grx,gry,gacc,eacc,gstr,estr,pl_ns,e2_ns,ok
0,0,0,0,0,64,0,10,10,0,0,806,4387,1
...
800,369,1897,122,321,288,1920,0,0,-6,-6,738,3741,1
# done valid=801 matches=801 acc_diff=0 str_diff=0 first_mm=-1
# pl_ns min=723 avg=739 max=806
# e2_ns min=3720 avg=3752 max=4387
# total_us=3012
# tick_cnt=801
```

Columns: `idx` is the sample index; `gx..gry` echo the state and reference written to the IP; `gacc/gstr` are the FPGA-returned commands against `eacc/estr`, the MATLAB-expected ones; `pl_ns` is the kick-to-done latency in the PL; `e2_ns` adds the AXI overhead; `ok` is 1 when both commands match. After '=== done; entering idle loop ===', stop the terminal log and rename the capture to `fpga_log.csv`. If the header line is missing because capture started late, prepend it manually.

Stage 6: Open-loop validation in MATLAB

Set MATLAB's working directory to the folder with `fpga_log.csv` and the eight `.dat` files, then run `compare_fpga_matlab`.

Expected console output:

```
parsed 801 data rows from fpga_log.csv
input-echo check : PASS
exp-echo check : PASS
FPGA vs MATLAB:
  matches      : 801 / 801 (100.00%)
  accel diffs  : 0      max|err|=0
  steer diffs  : 0      max|err|=0
PL-only latency (ns): min=723  avg=739.8  max=806
End-to-end latency (ns): min=3720 avg=3752.7 max=4387
```

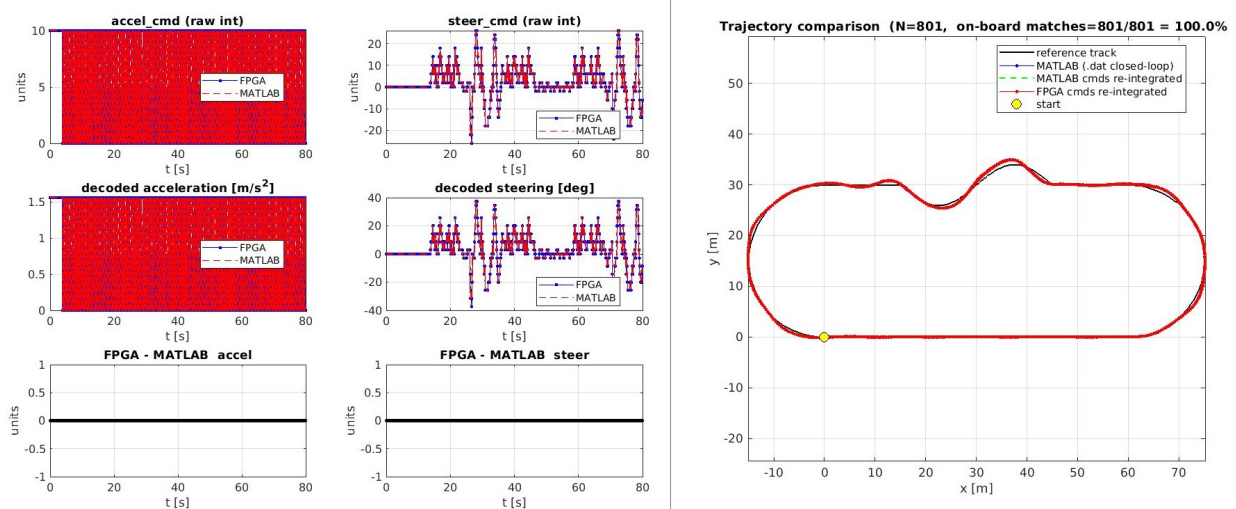


Figure 6.1: Open-loop validation output. Raw and decoded command comparisons overlap fully, the per-sample FPGA minus MATLAB difference is flat zero, and the trajectory overlay shows the reference track, the MATLAB closed loop, and both re-integrated command sequences in alignment.

Acceptance criteria: matches 801/801, zero accel and steer diffs, both echo checks PASS, all trajectory lines overlapping, and PL-only latency under 1 us average (the IP solves the problem in well under one MATLAB T_s of 100 ms). If something fails, troubleshoot in this order. An input-echo FAIL means the FPGA read back different state than the C code wrote: check the AXI register slicing against the map in Appendix A.1. An exp-echo FAIL means the `.dat` files do not match the `trajectory_data.h`

compiled into the app: re-run `fcs_mpc_v2_tb3.m` and rebuild. Matches below 100% mean the VHDL disagrees with the .m: run the Stage 3 testbench and fix the VHDL.

Stage 7: Closed-loop validation

7.1 Why the open-loop replay is not enough

The replay of Stages 5 and 6 proves per-sample equivalence, but only at the states MATLAB happened to visit, and every sample starts from a known-good recorded state, so an error cannot propagate. The closed-loop harness removes that safety net. The controller's own commands drive the plant, the next state depends on the previous command, and any disagreement between implementations compounds step after step. It validates the controller in the loop it will actually run in, and it exercises state regions that a diverging trajectory would create. A single flipped low-order bit anywhere would send the two trajectories visibly apart within a lap.

7.2 Harness architecture (`mpc_closed_loop.c` + `reference_track.h`)

One C file implements the whole experiment and compiles for two targets. On a PC it builds with the **SW backend**: a bit-exact C port of the controller (the same 64-entry quadrant LUT construction, the same option arrays in the same order, the same shifts, clamps, and strict less-than tie break as the .m and the VHDL). On the Cora it builds with the **FPGA backend**: the controller step becomes the same AXI sequence as the replay app (six input writes, kick, poll DONE, read two sign-extended sfix6 outputs). The choice resolves at compile time: an explicit `-DMPC_BACKEND_FPGA` or `-DMPC_BACKEND_SW` wins, otherwise `__arm__` selects FPGA and a host build selects SW. The software controller is compiled only when the SW backend is selected, so a closed-loop FPGA binary physically cannot obtain commands from software; they can only come from the IP over AXI. This matters for validation integrity (Appendix D).

`reference_track.h` is auto-generated from the MATLAB testbench and carries the 358-point reference track (`REF_X`, `REF_Y`), the simulation length (801 steps), the initial state (0,0,0,0), and every plant and scaling constant: $T_s = 0.1$ s, wheelbase 2.5 m, drag 0.99, steer decode 0.025 rad per unit, `SF_POS/SF_PSI/SF_V`, lookahead minimum 0.8 m and gain 0.25. MATLAB and C therefore read the same numbers from the same generated source, and a track change in the testbench propagates by regeneration, never by hand edits.

Each step the harness runs the same sequence as the MATLAB testbench: find the nearest track point (this also accumulates lap count and tracking error), walk the lookahead distance along the track, wrap `psi` to $[-\pi, \pi]$, quantize the state with round-half-away-from-zero saturating int16 conversion, call `mpc_step`, log a CSV row, then integrate the bicycle plant with the returned commands. The heading update computes $\tan$ as $\sin/\cos$ from the shared deterministic primitives below.

7.3 Determinism: making two doubles agree bit for bit

The open-loop path never needed floating point to agree across machines, because the plant lived only in MATLAB. In the closed loop both sides integrate the plant, so every double operation must produce the identical bit pattern on a PC running MATLAB, on a PC running the C harness, and on the bare-metal Cortex-A9. Library `sin`, `cos`, `sqrt`, and `round` do not guarantee that across implementations. The harness therefore uses no `libm` on the target and replaces every sensitive primitive:

- `sincos_det`: range reduction to $[-\pi, \pi]$ via a deterministic round, quadrant folding, then fixed degree-13 (`sin`) and degree-14 (`cos`) polynomials with hard-coded coefficients. Only IEEE-754 additions and multiplications in a fixed order, so MATLAB (mirroring the same polynomial) and C produce identical bits.
- `det_sqrt`: the ARM VFP hardware instruction `vsqrt.f64`, which is IEEE-correct (correctly rounded), so it matches `sqrt` on the host and in MATLAB exactly.
- `det_floor`, `det_round`, `det_lround`: integer-cast constructions that reproduce MATLAB's floor and round-half-away-from-zero semantics without `libm`.
- `q16`: quantization that rounds half away from zero and saturates, matching MATLAB's `int16()`.

Bare metal also lacks float `printf`, so the CSV writer formats doubles as integer and six-digit fraction parts by hand. On the host the same writer prints with `%.10g`.

7.4 Output format

The harness streams one CSV row per step (to a file on the host, over the UART on the Cora, where the capture is renamed `c_closed_loop.csv`):

```
step,x,y,psi,v,xq,yq,psiq,vq,rxq,ryq,accel,steer,err
...
# backend=FPGA steps=801 laps=1.804469 max_err=1.008131 m
```

Columns: the physical double state (`x`, `y`, `psi`, `v`), the quantized int16 state and reference actually handed to the controller (`xq`, `ryq`), the returned commands, and the distance to the nearest track point (`err`). The footer reports the backend, the lap count (wrap detection plus fractional progress), and the peak of the `err` column. Both footer values also derive from the logged rows themselves, so a capture stopped before the footer prints loses nothing. The quantized columns make every controller call

reproducible in isolation: any row can be replayed through `fcs_mpc_v2.m`, the SW port, the VHDL TB, or the FPGA, and all four must return the same pair.

7.5 Results

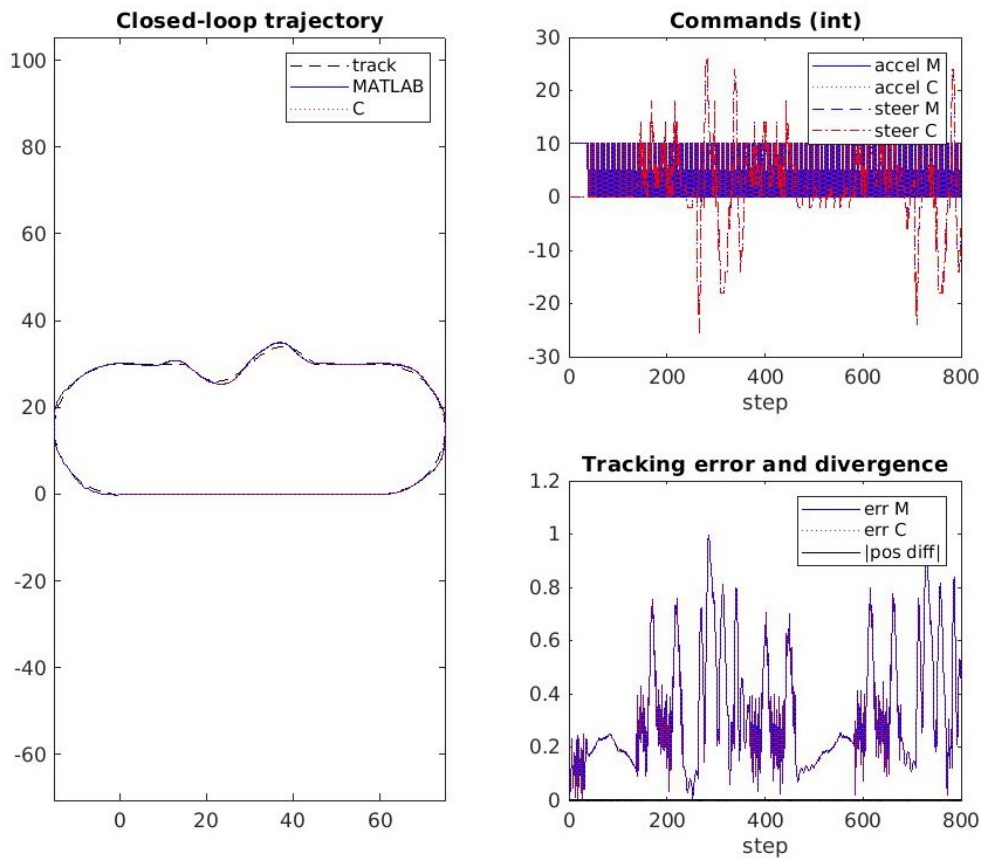


Figure 7.1: Closed-loop validation, produced by the comparison script from `matlab_closed_loop.csv` and the Cora UART capture `c_closed_loop.csv` (labelled C). Left: reference track with the two trajectories overlapping. Top right: acceleration and steering command sequences from both sides. Bottom right: per-step tracking error of each side and the absolute position difference between the two trajectories, which stays at zero.

The harness ran on the Cora with the FPGA backend, and the 801-row UART capture was compared against the MATLAB run column by column. Every integer column is identical at all 801 steps: the quantized states and references handed to the controller (`xq..ryq`) and both returned commands, with zero accel diffs and zero steer diffs. The double columns agree to within $5e-7$, which is exactly the rounding of the bare-metal 6-digit CSV formatter, and the position difference between the two trajectories stays below $1e-6$ m over the whole run. In other words, every digit the Cora printed matches the MATLAB value; the deterministic primitives of Section 7.3 left the print resolution as the only visible difference. The lap counter gives 1.80 laps on both sides. The commands are the IP's own by construction, since the FPGA binary contains no software controller (Section 7.2).

One number needs interpreting. The `err` column peaks at 1.008 m on both sides, while the Stage 1 testbench reports 0.987 m; both describe the same trajectory and differ only in reference discretization. The harness measures distance to the nearest of the 358 exported track vertices, spaced about 0.5 m, which upper-bounds the distance to the track itself. Measured against the continuous track polyline, the same trajectory peaks at 0.98 m, consistent with the testbench figure. The acceptance quantity for this stage is the divergence between implementations, not the absolute tracking error, and that divergence is zero.

Acceptance criteria for this stage: identical command and quantized-state columns at every step, per-step position difference at or below the print resolution ($1e-6$ m), and the same lap count on both sides. Any real divergence points at either a controller mismatch (run the Stage 3 TB) or a non-deterministic plant primitive (check that no `libm` call crept back in).

Stage 8: Performance vs C/C++ MPC solvers

For context, the table summarizes typical per-step solve times for the same class of problem (small, dense, embedded MPC with a candidate space or QP size around 10^2) in C/C++ on a comparable embedded target, the ARM Cortex-A9 at 666 MHz that sits next to the PL on the Cora. The figures are typical published ranges from the projects' own benchmarks and the embedded-MPC literature; they are wide because solve time depends on problem size, conditioning, warm starts, and compiler flags. Treat them as order-of-magnitude indicators, not contracts.

Implementation	Typical per-step solve time (Cortex-A9 class)
This work, FPGA enumeration (Zynq PL)	739.8 ns avg / 806 ns worst case
This work, same algorithm in C on the A9 PS (estimated)	about 100-500 us for the 136-candidate enumeration
OSQP (operator-splitting QP)	about 50 us - 5 ms
qpOASES (active-set QP)	about 100 us - 10 ms
HPIPM (interior point)	about 20-500 us
acados (code-generated MPC)	about 50-500 us
FORCES Pro (commercial)	about 50-500 us
CVXGEN (commercial code generator)	about 50 us - 1 ms

The FPGA solves the problem in about 740 ns per step; comparable software solvers on the same-class CPU need 50 us to several ms, roughly 70 to 1000 times longer. Two factors explain it: the FPGA evaluates all 136 candidates in parallel followed by a single argmin reduction, where software iterates; and the fixed-point arithmetic uses small, problem-tuned widths (8-16 bits) instead of 64-bit floats. The 3.75 us end-to-end average is dominated by about 3 us of AXI overhead (six writes, kick, polling, two reads); a single packed input write would be the next optimization. The comparison is most meaningful for this specific algorithm, an enumeration over a discrete option set with a kinematic simulator inside the cost. For continuous-action MPC with longer horizons and a true QP, OSQP, HPIPM, or acados become competitive in software, and an FPGA version would itself need a very different architecture, likely a fixed-iteration ADMM or interior-point pipeline.

Stage 9: Algorithmic comparison with the HDL Coder MPC of Purraji et al.

Purraji, Zamiri, and de Castro, 'Easy and Straightforward FPGA Implementation of Model Predictive Control Using HDL Coder' (Electronics 2025, 14, 419), implement an FCS-MPC for a grid-connected single-phase H-bridge converter, generate the VHDL automatically with MATLAB Simulink HDL Coder, and validate it with a hardware-in-the-loop model and the Vivado ILA on an ARTY Z7-20. Their controller and the one in this project belong to the same family: both are one-step finite-control-set MPCs that enumerate a discrete candidate set, evaluate a cost per candidate, and pick the argmin, with no QP and no iterative solver. Almost everything else differs, and the differences are instructive.

9.1 Side-by-side summary

Aspect	Purraji et al. 2025 (paper)	MATLAB fcs_mpc_v2.m (this work)	VHDL / FPGA (this work)
Plant	Grid-connected single-phase H-bridge (three-phase variant shown too)	Kinematic bicycle vehicle model	None on chip; the plant stays in MATLAB or on the PS
Controlled variable	Inductor current i_L injected to the grid	Vehicle position on a reference track	Same as MATLAB
Candidate set	3 switching states (+Vdc, 0, -Vdc); 8 for three-phase	$17 \times 8 = 136$ (steer, accel) pairs	Same 136, evaluated in parallel
Horizon	1 step	1 step	1 step
Prediction model	Forward-Euler scalar current update, $i_L(k+1) = i_L(k) + T_s/L (v_{inv} - v_g - R i_L)$	Nonlinear kinematic update needing sin/cos of heading	Same update via two 256-entry 8-bit LUTs and shifts
Cost function	Squared tracking error, $CF = (i_L(k+1) - i_{ref}(k+1))^2$; three-phase sums squares per phase	Clamped L1 position error $\times 15 + \text{steer} \times 2 - v_{next} \times 2$	Identical to MATLAB, bit for bit
Arithmetic	Fixed point via fixdt(S,T,F); 32-bit signals, multiplier inputs limited to 18 x 25 bits to fit one DSP48 each	int16 throughout, scale factors baked into shifts, no explicit fractional formats	signed 6-16 bit, shift-based scaling, small products only
Trigonometry	None needed (linear circuit model)	256-entry quarter-folded sin/cos LUTs, 8-bit amplitude	Same LUTs as VHDL constants
Design route	Simulink model, VHDL auto-generated by HDL Coder	Hand-written MATLAB int16 code	Hand-written VHDL translation of the .m
Execution model	Entire algorithm in a single clock period, registered I/O; Tclk,min = 30.125 ns	Sequential loops in software	Combinational candidate evaluation, registered I/O, 5-cycle kick-to-capture FSM at 50 MHz
Measured timing	500 ns sampling step in the experiment, 32 ns theoretically possible	n/a (desktop MATLAB)	723-806 ns per solve including AXI kick and polling granularity

Aspect	Purraji et al. 2025 (paper)	MATLAB fcs_mpc_v2.m (this work)	VHDL / FPGA (this work)
Resources (controller alone)	216 LUTs, 7 FFs, 8 DSPs on ARTY Z7-20	n/a	Small footprint on xc7z007s; no DSP-heavy multiplies
System integration	Standalone PL design, converter or HIL model on the same fabric	n/a	Memory-mapped AXI4-Lite peripheral under PS software control
Verification	Simulink simulation, then HIL with ILA waveform inspection	Closed-loop simulation against acceptance criteria	Bit-exact equivalence at three levels (xsim TB, open-loop hardware replay, closed loop)

9.2 What the differences mean

The candidate set is physical there and synthetic here. In the H-bridge the actuator is genuinely discrete: the converter has exactly three output voltage states, so FCS-MPC enumerates reality. In the vehicle problem the actuation is continuous, and the 136-pair grid is a design choice that discretizes it. That changes the engineering trade: the paper cannot change its set size, while this project trades grid density against tracking quality and combinational area. It also changes the cost: with three candidates a squared error costs one multiplier and is the natural choice; with 136 parallel evaluations a squaring multiplier per candidate is expensive, which is one reason this design uses the clamped L1 norm plus separate effort and speed terms.

The prediction step is linear there and nonlinear here. The paper's model is one multiply-accumulate per candidate, so the whole controller fits in a 30 ns critical path. The bicycle model needs sin and cos of the candidate heading, which is where the quarter-folded LUTs, the modulo-256 heading wrap, and the shift-based rescaling come from. Nothing comparable exists in the paper because a linear circuit never leaves the affine world.

HDL Coder against hand-written VHDL. The paper's central claim is that HDL Coder reaches synthesis quality comparable to hand-coded HDL while removing the HDL expertise barrier, and its numbers support that for this problem class. This project still wrote the VHDL by hand, for a reason the paper does not face: the goal was bit-exact equivalence to an already-existing int16 MATLAB reference, including its tie-break order, its saturation semantics, and its exact shift behavior. HDL Coder generates correct hardware from a Simulink fixed-point model, but matching a pre-existing hand-written .m bit for bit, ties included, is simpler to guarantee when every line of the translation is explicit. The two approaches optimize different things: development speed and accessibility there, verifiable numerical identity here.

Single-cycle against FSM. Both implementations compute the candidate evaluation combinationaly with registered I/O. The paper pushes the whole algorithm into one clock period and lets the clock constraint discover the critical path (30.125 ns). This design wraps the combinational core in a small kick/run/capture FSM at a fixed 50 MHz, spending five cycles per solve, because the controller lives behind an AXI handshake rather than free-running next to the plant. The 740 ns measured here against the paper's 500 ns experimental step is therefore not a like-for-like race; the problems differ by a factor of 45 in candidate count, and this design's latency is dominated by handshake granularity, not by the arithmetic.

Validation philosophy. The paper validates behavior: HIL waveforms on the ILA show the controlled current tracking its reference. This project validates identity: the same integers out of MATLAB, xsim, and silicon at every one of 801 samples, and in Stage 7 the same double-precision trajectory bit for bit. Behavioral validation is the right bar for a power converter whose reference design is the Simulink model itself. Identity is the right bar here because the MATLAB code is the specification and any silent numerical drift would invalidate the offline tuning.

Appendix A: Tricky fixes and known pitfalls

These were discovered during the project and are listed so they do not have to be rediscovered.

A.1 AXI register map

Offset	Name	Access	Purpose
0x00	CTRL	RW	Bit 0 = KICK (self-clearing), bit 1 = EN, bit 2 = IRQ_EN
0x04	STATUS	RO	Bit 0 = DONE, bit 1 = BUSY
0x10	X	RW	State x (sfix14, low 14 bits)
0x14	Y	RW	State y (sfix13)
0x18	PSI	RW	State psi (sfix9)
0x1C	V	RW	State v (ufix9)
0x20	REF_X	RW	Reference x (sfix14)
0x24	REF_Y	RW	Reference y (ufix12)
0x30	ACCEL	RO	Output accel_cmd (sfix6, sign-extended to 32 bits)

Offset	Name	Access	Purpose
0x34	STEER	RO	Output steer_cmd (sfix6, sign-extended)
0x40	TICK	RO	Solve count, increments on each DONE

These offsets are the MPC_OFF_* macros in mpc_smoketest.c and the R_* macros in mpc_closed_loop.c, and they must match the slice expressions in the S00_AXI case statements. If the C app reads a non-zero offset and gets zeros, suspect a mismatch.

A.2 sfix5 to sfix6 widening of accel_cmd

ACCEL_OPTS includes -20, which does not fit sfix5. The widening has to land in all five of these places:

File	What to change
fcs_mpc_v2_fixpt.vhd	Entity port: accel_cmd : OUT std_logic_vector(5 DOWNTO 0)
fcs_mpc_v2_fixpt.vhd	Internal best_acc : signed(5 DOWNTO 0)
MPC_controller_AXI_v1_0_S00_AXI.vhd	Internal signal: mpc_accel : std_logic_vector(5 DOWNTO 0)
MPC_controller_AXI_v1_0_S00_AXI.vhd	Sign extension: slv_reg12(31 DOWNTO 6) <= (others => mpc_accel(5))
MPC_controller_AXI_v1_0_S00_AXI.vhd	Data slice: slv_reg12(5 DOWNTO 0) <= mpc_accel

Symptom of a partial fix: -20 (and possibly -5, -10) wrap to positive values in the FPGA output, and mismatches concentrate where braking is optimal. The C side needs no change; reg_to_s16 already sign-extends correctly.

A.3 .dat file truncated bit widths

The .dat files store each value as 4 hex digits of the uint16 bit pattern, but the underlying value only uses N bits matching the AXI port width (x: 14, y: 13, psi: 9, v: 9, ref_x: 14, ref_y: 12, accel: 6, steer: 6). The comparison script must sign-extend from those widths. A parser that assumes plain 16-bit signed misreads negative narrow values: psi = -30 stored as 482 in 9-bit storage reads as +482.

A.4 mpc_axi_wrapper.vhd is an orphan

Earlier iterations sliced the AXI registers in a separate wrapper layer. That layer is absorbed into the S00_AXI file. The old wrapper is in no file group and no hierarchy; if it reappears, remove it, or it causes width-mismatch errors at elaboration.

A.5 Vivado wrong part default

Vivado defaults to the last project's part, often xc7z020. Synthesis succeeds for the wrong part but the bitstream targets the wrong silicon. The Cora Z7-07S is xc7z007s-clg400-1. Fix in Tools -> Settings -> General -> Project Device, then Reset Runs and regenerate.

A.6 Boot banner lost

UART output begins the moment the FSBL hands off. A terminal connected late loses the header line and the boot smoketest result. Always open the terminal first, then launch.

A.7 File paths on Windows

Vivado and Vitis 2022.1 have intermittent problems under OneDrive, Dropbox, or any sync service (file locks during synthesis, e.g. 'boost::filesystem::remove: file is being used by another process' on simulate.log) and with paths past the 260-character limit. Keep projects in a short, sync-disabled path like C:\work\mpc_vhdl.

A.8 Stale wave configuration

After changing the simulation top, Vivado reuses the previous .wcfg even when the signal names no longer exist. The wave window shows the new TB apparently running with only empty orphaned slots. Fix: close_wave_config in the Tcl console, delete the .wcfg from the project root, restart simulation.

A.9 Vitis 'No Platform exists'

After Update Hardware Specification with a structurally changed XSA, Vitis sometimes leaves the platform metadata inconsistent and builds fail with 'No Platform exists with name: coraz7_07s_plat'. Cleanest fix: create a new platform under a new name from the new XSA and a new app on top. The XPAR_* references still resolve because they come from BD instance names, not platform names.

A.10 Software Driver group breaks the BSP build

The IP wizard creates a 'Software Driver' group with empty .c files; the BSP Makefile's *.c wildcard then fails with 'cc1.exe: fatal error: *.c: Invalid argument'. Remove the file group in the IP Packager. The apps use Xil_In32/Xil_Out32 directly, so the driver layer was unused anyway.

Appendix B: Iteration workflows

B.1 Trajectory-only change (initial conditions, reference path, lookahead, sim length): edit fcs_mpc_v2_tb3.m, run it to regenerate all artifacts, copy the new trajectory_data.h (and reference_track.h) into the app sources, rebuild in Vitis, reprogram, capture, compare. No Vivado work; about 3 minutes.

B.2 Algorithm parameter change (weights, option sets, scales, clamps): edit fcs_mpc_v2.m and make the equivalent edits in fcs_mpc_v2_fixpt.vhd (and the SW port in mpc_closed_loop.c). Run the MATLAB testbench and check tracking quality first. Run the Stage 3 VHDL TB; it must pass 801/801 before any synthesis. Then bitstream, XSA, platform update, rebuild, reprogram. About 15 minutes; synthesis dominates.

B.3 Controller I/O structural change (ports, widths, new AXI registers): edit the DUT and the S00_AXI file consistently, follow the A.2 widening procedure per affected signal, re-package the IP, regenerate output products on the BD, and continue from bitstream generation. New registers also mean updating the C-side offset macros and the A.1 map. About 25 minutes.

Appendix C: Auto-generated artifacts and the validation script

C.1 design_1.bd

Vivado stores the block design as JSON and regenerates it on every save, so the shareable form is a Tcl export (File -> Export -> Export Block Design), replayed with vivado -mode batch -source design_1.tcl. The verified BD configuration to match when recreating by hand: processing_system7_0 (xilinx.com:ip:processing_system7:5.5) with PCW_USE_M_AXI_GP0 = 1, PCW_USE_FABRIC_INTERRUPT = 1, PCW_IRQ_F2P_INTR = 1, FCLK_CLK0 = 50 MHz, UART1 on MIO48/49; ps7_0_axi_periph (axi_interconnect:2.1) with one slave and one master; rst_ps7_0_50M (proc_sys_reset:5.0) at defaults; and the custom MPC_controller_AXI:1.0 with no parameters. The address section maps SEG_MPC_controller_AXI_0_S00_AXI_reg at 0x43C00000, range 64K, which is what produces the XPAR_* base address in xparameters.h.

C.2 Constraint files

No user .xdc is needed. The Cora board preset applies pin and clock constraints, and Vivado auto-generates two helpers. dont_touch.xdc preserves hierarchy per IP (KEEP_HIERARCHY SOFT on each instance) so debug tools can find named cells; nothing in it is editable, and it regenerates if stale. design_1_auto_pc_0_ooc.xdc declares create_clock -period 20 on the AXI protocol converter for out-of-context synthesis; the 20 ns period equals the 50 MHz FCLK_CLK0, and checking that equality is a cheap way to confirm the BD clock setup. A custom .xdc becomes necessary only for extra I/O pins (PMOD, Arduino headers), a clock other than the 50 MHz preset, or an external PL clock input; none apply here.

C.3 compare_fpga_matlab.m

The script parses fpga_log.csv into per-sample vectors, reloads the eight .dat files with bit-width-aware decoding, runs the input-echo check (the FPGA's echoed inputs against what the C code wrote) and the exp-echo check (the expected commands in the CSV against the .dat ground truth), prints diff statistics and latency, and plots the Figure 6.1 composite. The key non-obvious part is the DAT_SPEC struct that records each field's truncated width and signedness; without it, negative narrow fields decode wrong (Appendix A.3).

```
DAT_SPEC = struct( ...
    'x', {{'x.dat', 14, true}}, 'y', {{'y.dat', 13, true}}, ...
    'psi',{{'psi.dat', 9, true}}, 'v', {{'v.dat', 9, false}}, ...
    'rx', {{'ref_x.dat',14,true}}, 'ry', {{'ref_y.dat',12,false}}, ...
    'acc',{{'accel_cmd_expected.dat', 6, true}}, ... % was 5 (sfix5), now sfix6
    'str',{{'steer_cmd_expected.dat', 6, true }});
```

The acc width was updated from the legacy 5 to 6 after the Appendix A.2 widening. The old value never affected the verified result because the test trajectory never selected -20, but a future trajectory that does would sign-extend as +12 under the old decoding and produce a false negative on exp-echo.

The trajectory overlay comes from a small sim_plant helper that forward-integrates the bicycle model from the origin using each side's command sequence, with the same constants as the testbench (L = 2.5 m, Ts = 0.1 s, drag 0.99, steer decode 0.025). Re-integration from scratch makes the equivalence visible: identical commands must re-integrate to identical trajectories, which is what the overlapping traces show.

```
function [xs, ys] = sim_plant(acc_int, str_int, SF_V, L, Ts)
    N = numel(acc_int);
    xs = zeros(N, 1); ys = zeros(N, 1);
    x = 0; y = 0; psi = 0; v = 0;
```

```

for k = 1:N
    acc = acc_int(k) / SF_V;
    str = str_int(k) * 0.025;
    x = x + v * cos(psi) * Ts;
    y = y + v * sin(psi) * Ts;
    psi = psi + (v / L) * tan(str) * Ts;
    v = (v + acc) * 0.99;
    xs(k) = x; ys(k) = y;
end
end

```

The script also rebuilds the reference track with its own `build_reference_track` helper so the reference plots as a clean line independent of the jumpy per-sample lookahead values; if the track geometry changes in the testbench, that helper must change to match, or the reference line in the figure will look right but be subtly wrong.

C.4 Files you still need to bring

On a clean machine, only the source files are needed: `fcs_mpc_v2.m`, `fcs_mpc_v2_tb3.m`, `fcs_mpc_v2_fixpt.vhd`, the two AXI wrapper files, `fcs_mpc_v2_fixpt_tb.vhd`, `mpc_smoketest.c`, `mpc_closed_loop.c`, and `compare_fpga_matlab.m`. The BD is recreated from Stage 2 or a Tcl export, the XDCs regenerate automatically, and `trajectory_data.h`, `trajectory_data_pkg.vhd`, and `reference_track.h` regenerate from one testbench run.

Appendix D: Validation integrity, is the 100% match real?

A 100% match is a strong claim, so this appendix addresses the natural concern that the result could be an artifact of the validation pipeline rather than evidence of equivalent computation. Six plausible ways the pipeline could produce a false match were enumerated and checked.

Vector	Mechanism	Status
D.1.a	VHDL DUT contains a sample-indexed LUT of expected outputs	Structurally impossible (D.2)
D.1.b	AXI wrapper bypasses the DUT and returns hardcoded values	Verifiably false (D.2)
D.1.c	C app prints the expected value in the 'got' column	Verifiably false (D.2)
D.1.d	<code>verbose_to_csv.py</code> synthesizes matching rows for missing samples	Real risk, bypassed by the current flow (D.3)
D.1.e	Output register mis-routed to echo an input register	Ruled out empirically (D.4)
D.1.f	Comparison script treats missing samples as matches	Verifiably false (D.2)

D.2 Per-file audit

Every FPGA-related file was audited at line level with concrete grep queries and data-flow tracing, so the checks can be re-run by any reader.

File	Audit result
<code>fcs_mpc_v2_fixpt.vhd</code>	Clean. The entity has only <code>clk</code> , <code>reset</code> , the six state inputs, and the two command outputs; no sample-index input exists, so the DUT cannot know which sample is driven. <code>grep</code> for <code>accel_exp</code> , <code>steer_exp</code> , <code>trajectory</code> , <code>sample</code> , <code>idx</code> returns nothing. The output is a pure function of the current state inputs.
<code>MPC_controller_AXI_v1_0_S00_AXI.vhd</code>	Clean. The AXI write case statement covers <code>slv_reg0..9</code> and falls to 'when others => null', so the output registers <code>slv_reg12/13</code> are unreachable from the bus. Only the <code>S_CAPTURE</code> state writes them, copying from <code>mpc_accel/mpc_steer</code> , which come straight from the DUT port map. No alternative path exists.
<code>mpc_smoketest.c</code>	Clean. <code>accel_exp/steer_exp</code> appear three times: the <code>#include</code> and two read-only assignments to local comparison variables used in the equality test and as separate CSV columns. The AXI write loop writes only the state arrays; the expected arrays never reach the bus, so the 'got' and 'expected' columns come from textually different sources.
<code>trajectory_data.h</code>	Clean. Eight static <code>const int16_t</code> arrays and one <code>#include</code> ; no macros, functions, or conditional compilation.
<code>compare_fpga_matlab.m</code>	Clean. The row count comes from actual lines read from <code>fpga_log.csv</code> ; a truncated log reports a smaller N, never an inflated percentage. The match count is <code>nnz</code> of the <code>ok</code> column, which the FPGA side computed, and the script independently re-derives the diff vectors from the <code>got/expected</code> columns as a cross-check.
<code>MPC_controller_AXI_v1_0.vhd</code>	Clean. Pure AXI port plumbing around the <code>S00_AXI</code> instantiation, no data path of its own.

The key checks, verbatim, so they can be repeated. The DUT structural check and its entity (there is no sample-index port, so the DUT cannot know which sample is being driven; the output is a pure function of the state inputs):

```
$ grep -niE "accel_exp|steer_exp|trajectory|sample|idx" fcs_mpc_v2_fixpt.vhd
(no output)
```

```
PORT( clk      : IN  std_logic;
      reset    : IN  std_logic;
      x        : IN  std_logic_vector(13 DOWNTO 0);
      y        : IN  std_logic_vector(12 DOWNTO 0);
      psi      : IN  std_logic_vector( 8 DOWNTO 0);
      v        : IN  std_logic_vector( 8 DOWNTO 0);
      ref_x    : IN  std_logic_vector(13 DOWNTO 0);
      ref_y    : IN  std_logic_vector(11 DOWNTO 0);
      accel_cmd : OUT std_logic_vector( 5 DOWNTO 0);
      steer_cmd : OUT std_logic_vector( 5 DOWNTO 0) );
```

The wrapper's only paths into the output registers (the AXI write case ends in null, so slv_reg12/13 are unreachable from the bus; only the capture state loads them, from the DUT outputs):

```
case loc_addr is
  when "00000" => slv_reg0 <= S_AXI_WDATA;
  ...
  when "01001" => slv_reg9 <= S_AXI_WDATA;
  when others => null; -- slv_reg12/13 fall here
end case;

-- S_CAPTURE state, the only writer of the output registers:
slv_reg12(31 downto 6) <= (others => mpc_accel(5));
slv_reg12(5 downto 0)  <= mpc_accel; -- from the DUT port map
slv_reg13(31 downto 6) <= (others => mpc_steer(5));
slv_reg13(5 downto 0)  <= mpc_steer;
```

And the C-side source separation in mpc_smoketest.c (the 'got' column comes from an FPGA register read, the 'expected' column from the header array; if both came from one source, every match would be tautological):

```
uint32_t got_accel_raw = mpc_read(MPC_OFF_ACCEL); /* from the FPGA register */
int32_t  got_accel     = reg_to_s16(got_accel_raw);
int32_t  exp_accel     = (int32_t)accel_exp[t];  /* from the .h array */
```

D.3 The verbose_to_csv.py caveat

The legacy post-processing script for verbose mode does contain code that synthesizes a matching row when a sample is missing from the UART log, filling it with the expected values and counting it in `n_inferred`. Used with an incomplete log it would fabricate matches, visibly (the counter prints) but fabricate nonetheless. The verified result was produced in `MODE_CSV`, where the C app emits CSV directly and this script is not in the loop; the UART capture is itself `fpga_log.csv`, and no inferred rows exist in that path. Anyone returning to verbose mode should treat any nonzero `n_inferred` as disqualifying.

D.4 The output-wiring question and its empirical answer

The boot-time smoketest writes a sentinel to `REG_X` and reads it back, which verifies the input half of the address map but not that offset `0x30` routes to `mpc_accel` rather than, say, `slv_reg4`. Four observations rule the confusion out empirically. Reads at `0x30` land exactly in the `ACCEL_OPTS` set, not in the range of `x` values (which span `-960` to `4800` on this trajectory). Reads at `0x34` land exactly in `STEER_OPTS`. The `TICK` register reaches exactly `801`, matching the number of `S_CAPTURE` transitions a computing FSM performs. And the per-sample PL latency of `723-806 ns` is consistent with the 5-cycle kick/run/capture FSM at `50 MHz` plus polling granularity; a passthrough would be faster and flatter. A one-line hardening (writing a unique sentinel to every input register and reading each back) closes the gap completely; Section D.7 describes the fielded version.

D.5 Optional hardening: fresh-state sentinel test

A further check, beyond what the current pipeline provides, is to query the IP once at boot with state values that appear nowhere in `trajectory_data.h`, so trajectory-data hardcoding could not help even in principle:

```
/* Anti-hardcoding sentinel, run once at boot before trajectory replay */
mpc_write(MPC_OFF_X,      (uint32_t)(int32_t)500);
mpc_write(MPC_OFF_Y,      (uint32_t)(int32_t)200);
mpc_write(MPC_OFF_PSI,    (uint32_t)(int32_t)0);
mpc_write(MPC_OFF_V,      (uint32_t)(int32_t)100);
mpc_write(MPC_OFF_REF_X,  (uint32_t)(int32_t)600);
mpc_write(MPC_OFF_REF_Y,  (uint32_t)(int32_t)200);
mpc_write(MPC_OFF_CTRL, MPC_CTRL_KICK | MPC_CTRL_EN);
while (!(mpc_read(MPC_OFF_STATUS) & MPC_STATUS_DONE));
```

```
int32_t acc = reg_to_s16(mpc_read(MPC_OFF_ACCEL));
int32_t str = reg_to_s16(mpc_read(MPC_OFF_STEER));
xil_printf("[sentinel] x=500 y=200 v=100 -> acc=%d str=%d "
           "(MATLAB-expected acc=10 str=0)\r\n", (int)acc, (int)str);
```

With these inputs the vehicle sits at about (7.8 m, 3.1 m), heading 0, at 1.56 m/s, with the reference 1.56 m straight ahead; the optimal action is to accelerate. Running `fcs_mpc_v2` in MATLAB on the same inputs gives the ground truth (`acc=10, str=0`), a value never used during replay.

D.6 The strongest confirmations: xsim and the closed loop

The most compelling evidence is independent agreement across mutually exclusive toolchains. The VHDL behavioral testbench passes 801/801 while using no C application, no AXI wrapper, no synthesis, and no hardware; the FPGA passes 801/801 through the full pipeline. For both to be wrong they would need coordinated bugs in disjoint parts of the toolchain. The Stage 7 closed loop adds a third, harsher confirmation: the FPGA-backend harness physically contains no software controller (it is compiled out), so its commands can only come from the IP, and a single wrong command at any step would bend the trajectory away from MATLAB's for every subsequent step. The verified Cora capture shows zero command differences and trajectories matching to the UART print resolution over all 801 steps: the trajectory does not bend. Verdict: the 100% match is real, deterministic equivalence between the MATLAB algorithm and the FPGA implementation; no plausible hardcoding pathway exists in the current setup.

D.7 Hardening: input-echo as a true bus integrity check

One naming subtlety remained on the input path: the original `gx.gry` CSV columns were host-side echoes of the arrays, not values read back from the FPGA, so the MATLAB 'input-echo check' was really a host-side `.h` to `.dat` consistency check. It was not a hole (wrong inputs would have produced wrong outputs and failed the match), but the name promised more than it delivered. The fix reads the six input registers back over AXI after the outputs (placed after the latency timestamp so the diagnostic reads do not inflate `e2_ns`) and writes those read-backs into columns 2-7. The CSV schema is unchanged; only the meaning changes. The two checks now have distinct guarantees: input-echo passes when the FPGA register file held what was written (catching bus glitches, misrouting, and bad slicing), and exp-echo passes when the `trajectory_data.h` in the app and the `.dat` files in MATLAB came from the same testbench run (catching stale-rebuild bugs). On failure the script also localizes the first differing field and sample. Together the pipeline now verifies five layers: input-bus integrity, host-side data consistency, output computation correctness, output-bus integrity (a miswired `0x30` would not produce values inside `ACCEL_OPTS`), and FSM progress (`TICK = 801`). Any single failure visibly drops the match rate.

Notes

This procedure has been verified end to end on hardware. The open-loop criterion of 801/801 samples and the closed-loop criterion of zero trajectory divergence are both achievable and represent a fully deterministic correspondence between the MATLAB simulation and the FPGA implementation: same algorithm, same data, same arithmetic, same outputs at every sample. The pipeline is designed to fail loudly at each stage rather than silently; when something stops matching, the failure usually points at the boundary between two stages, most often a stale artifact or a missing rebuild. Appendix A covers the recurring causes.